

Minsoo Kim



+8210-7714-9201



vibrio0102@gmail.com



[linkedin](#)



[github](#)

Core Skills

Designing, building and operating an MLOps platform on on-prem Kubernetes:

I codified the entire path — ingestion → lakehouse → transformation → lineage → distributed training → experiment / model registry — on bare-metal Kubernetes in **Pulumi**, and I build and operate it. It runs **10 nodes / 235 running pods / 23 internal services** in production, and I worked across all 12 stacks — networking (Cilium), storage (SeaweedFS), identity (Dex OIDC → Kubernetes RBAC) — authoring 93% of stack-code changes and 83% of the ETL repository's commits. Distributed training, experiment tracking and the model registry — the segment closing the lifecycle — have been underway **since Aug 2026**, and when the bottleneck sits outside the code I work **down to the hardware**: replacing cabling and switches, fitting reclaimed RAM and NVMe into nodes.

Data pipelines and the lakehouse:

Designed and implemented a medallion pipeline over **Mar-Jul 2026 (5 months)** on **Airflow 3.x + dbt (Cosmos) + Apache Iceberg** with lineage auto-recorded in **DataHub**. Previously ran large-scale cloud pipelines on AWS EMR / PySpark.

Working alongside AI coding agents — with verification gates behind the speed:

I run AI coding agents continuously to lift **one person's throughput to team scale** (678 merged PRs in 6 months), and keep that speed honest with **719 SDK unit tests**, CI gates and pre-commit hooks that block tag/version drift and private-key commits, and **23 incident write-ups and 30 PRDs**. Earlier, built deduplication systems and chatbot prototypes with LangChain and embedding models.

Full-stack breadth and team leadership:

A path from iOS through backend (FastAPI / Flask), frontend (Next.js) and cloud (AWS / GCP) let me lead a data engineering team; today I design and enforce the split where **researchers own domain ETL and the platform owns execution, lineage and verification**.

Technical Skills

Languages: Python | Swift | JavaScript | Objective-C | Shell | SQL

Frameworks: PySpark | Flask | FastAPI | Kafka | Langchain | OpenAI | ReactJS | NextJS | Bun

laC & Platform: Pulumi | Kubernetes(kubeadm) | Cilium | SeaweedFS | Dex/OIDC | Docker

Data: Airflow | dbt(Cosmos) | DataHub

Lakehouse & OLAP: Apache Iceberg | DuckDB | ClickHouse

Database: MariaDB | PostgreSQL | PGVector | Firestore | Neo4J

ML: Ray/KubeRay | MLflow

Observability: Prometheus | Grafana | Loki | Vector

LLM: Langchain | LangGraph | OpenAI(Text Embedding) | RAG(Neo4J, PostgreSQL)

Data Analytics: CytoscapeJS | SigmaJS

iOS: ReactorKit | Async Await | Alamofire | Snapkit | Crashlytics | Kingfisher | SceneKit | SPM | React-Native

Cloud: AWS (EC2, Lambda, S3, IAM, EMR, Route53, RDS Aurora) | Firebase (Firestore, Storage) | GCP | Ubuntu | uv

Career Summary

Data Engineer (approx. 3 years): Designed, built and optimized large-scale data pipelines on AWS. Led data-platform and MLOps pipeline work with PySpark, Airflow and vector databases.

Backend Engineer (cumulative approx. 2-3 years): Built API servers and internal management systems in Python (Flask, FastAPI), including real-time WebSocket communication and Firebase integration.

Frontend & Client (approx. 4 years): Built data-visualization dashboards and BI tools with Next.js. Started out as an iOS developer, designing ReactorKit-based reactive architectures and shipping native apps.

Experiences

HUINNO

Data Engineer

Dec 2025 - Present

Seoul

- **Designed, built and operate the company's entire MLOps pipeline** (ingestion → lakehouse → transformation → lineage → distributed training → experiment / model registry). **Sole owner of cluster construction on a two-person platform team** — networking, storage, catalog, lineage, orchestration, training and identity: worked across all 12 Pulumi stacks. Operated **10 nodes / 235 running pods / 23 internal services** in production over 6 months, shipping 678 merged PRs and 1,204 commits, and codified bare-metal **Kubernetes** end-to-end in **Pulumi** for reproducible deploys.
- **Designed and implemented every branch of the ETL pipeline** (Mar-Jul 2026, 5 months; separate repository; 1,599 commits). Wrote the shared cross-product framework myself, along with a product pipeline, the MIMIC-IV pipeline and operational maintenance, and designed the remaining products to extend that framework — establishing a split where **researchers own domain ETL while the platform owns execution, lineage, verification and retries**.
- **[Data · Mar-Jul 2026] Removed the single-person dependency in dataset preparation**. Previously a researcher had to ask one data scientist, who ran a hard-coded script over SSH for **hours** and handed back a file — work stalled whenever that person was away, and reproducibility, lineage and versioning were all absent. Moved Bronze / Silver / Gold transformation to **scheduled execution with Airflow 3.x + dbt (Cosmos) + Apache Iceberg** and auto-recorded table-level lineage in **DataHub**, so researchers query directly instead of requesting and waiting.
- **[Training · Aug 2026-present] Built the distributed-training foundation to replace single-GPU manual runs**. Unified **10 GPUs** of differing models into one **KubeRay** pool behind a **single worker image** with pinned CUDA / PyTorch / ONNXRuntime and an internal SDK submission API (automatic checkpointing and mid-run resume supported). **Adoption is in progress; migrating research code remains open** — the two items below are the measurement work that identified what blocks it.
- **[Design · Aug 2026-present] Quantified the existing research workflow and designed a reproducibility architecture from the findings**. Surveyed the research repository and measured **51 remote branches, 67-88% of notebook cells carrying stored outputs, and experiment records isolated in local folders**; identified the root cause as not missing tooling but **results having no coordinates**. Designed and implemented a reproducibility contract that stamps **version, content hash, code commit, environment lock and seed** onto a dataset at execution time (freeze tool, manifest schema, integrity-verification CLI), positioning MLflow tracking / model registry (S3 artifacts, SSO, DataHub mirror) as the store for those coordinates.
- **[Foundation] Cut training-data delivery time 25×** — redesigned network and storage from **93.7 Mbps to 2.5 Gbps** (iperf3 measured), taking an 8 TB transfer from **8.4 days to 7.8 hours**. Applied **NVMe / HDD tiering** to an Iceberg lakehouse on **SeaweedFS** (S3-compatible), and opened Iceberg to ad-hoc SQL via **PostgreSQL + pg_duckdb**, replacing hand-written pyiceberg code with **a single line of browser SQL**.
- **[Hardware] Fixed the bottleneck below the software layer**. The 25× gain above came from tracing the cause **down to the physical layer**: I replaced a **faulty Ethernet cable**, specified and purchased the **switch** the workload needed, and **fitted reclaimed idle RAM and NVMe drives into nodes** to raise their specs. From cabling and components up to **Kubernetes network overhead**, I solve it in **both hardware and software**.
- **[Access] Consolidated shared passwords and unauthenticated endpoints into one identity**. Applied SSO across **23 internal services** with **Dex (GitHub OIDC) + oauth2-proxy** and carried the same identity through to **Kubernetes RBAC**. Replaced direct SSH into GPU servers with **browser-based JupyterHub development environments** (per-user volumes, egress-policy isolation, one-command bootstrap).
- **[Judgment] Reversed a GPU procurement decision with measurement**. Asked to evaluate expansion, measured 10 days with DCGM / Prometheus and found **5.3% utilization with zero queued jobs** — the bottleneck was not capacity but **1,112 GPU-hours held idle by dormant sessions** (91% of all held time) — and redirected the decision from purchase to reclamation.
- **[Method] Ran AI coding agents continuously to lift one person's throughput to team scale, while backing that speed with verification gates**. Delivered the above in 6 months alongside **719 SDK unit tests, CI gates and pre-commit hooks** that block tag/version drift and private-key commits, and **23 incident write-ups and 30 PRDs** that turn failures into prevention procedures.

Cellkey

Manager, Data Engineer

Jul 2025 - Dec 2025

Seocho-gu, Seoul

- Designed a data-cleaning pipeline on **Apache Airflow 3.x** and deployed it locally.
- Maintained and tuned **AWS Lambda, Batch and Step Functions** to carry scalable data workflows.

Netmelon

Mar 2025 - Jun 2025

Back-End Engineer

Freelance

- Implemented Android & iOS In-App Purchase API (subscriptions, renewals, cancellations)
- Packaged a Flask web server as a Docker image and deployed it on Google Cloud Run
- Implemented a feature to verify ownership of Firebase resources
- Wrote unit test cases for LLM-driven API routes, services, and functions
- Operated Firebase Firestore and Cloud Storage
- Refactored legacy code

Eazel Inc.

Jun 2022 - Sep 2024

Team Lead, AI Labs

Jung-gu, Seoul

- Owned the art-market ETL pipeline as team lead — architecture, deployment, monitoring and performance tuning
- Built dashboards based on Flask/NextJS
- Developed and operated auction data scrapers
- Built Airflow pipelines that collect, clean and load data on a schedule
- Built cleaning and loading pipelines with PySpark on AWS EMR
- Built a deduplication system on OpenAI and Facebook embedding models, then raised its accuracy and speed with a vector DB
- Managed AWS services (EC2, IAM, S3, Route53, RDS, EMR)
- Implemented CI/CD using Github Actions
- Built a chatbot prototype with LangChain and Streamlit
- Led the implementation of a Neo4j Cypher query tool and a PostgreSQL query tool for the chatbot
- Built the scraping layer on LangChain with information-extraction prompts

Eazel Inc.

Oct 2021 - Jun 2022

MLOps Engineer, MLOps

Jung-gu, Seoul

- Ran the full cycle from data analysis through modeling to validation, using HuggingFace NLP among other tooling
- Developed and operated Instagram data scrapers
- Built a social-network analysis dashboard on Flask / NextJS / CytoscapeJS and surfaced the findings that fed a Series A raise
- Wired the ML lifecycle through DVC
- Managed the data in PostgreSQL

Eazel Inc.

Jan 2019 - Oct 2021

iOS Developer, iOS Team

Gangnam, Seoul

- Led the design of the API client and its backend integration around iOS app performance and scalability
- Introduced CI/CD with Github Actions
- Introduced ReactorKit-based reactive programming and led the architecture behind it, improving app responsiveness and maintainability

Finple Inc.

Jan 2018 - Nov 2018

Software Engineer, DevOps

Yeongdeungpo, Seoul

- Maintained hybrid iOS and Android apps
- Developed a NodeJS-based data scraper

Projects

Eazel Inc.

Jun 2022 - Sep 2024

Data Engineer, AI Labs

Jung-gu, Seoul

Development and Operation of Data Pipeline for Price Database Service

Jan 2023 - Oct 2023

- Built the pipeline on one AWS Aurora RDS to keep development cost down.
- Processed and loaded data with PySpark on AWS EMR.
- Implemented partitioning and data optimization for distributed processing.

- Key contribution to Series B funding by leading the overall design and construction of the data platform (Price Database/MLOps).

Development and Operation of Art Market Data Scraper

Jan 2023 - Feb 2023

- Built a scraper with requests and BeautifulSoup4.
- Built a fast scraper on multithreading.
- Used LangChain information-extraction prompts to scrape into a fixed structure.
- Developed a social relationship collection scraper written in Swift, operating in the iOS environment.
- Ran 50 scraper instances at once through boto3.
- Controlled those instances over a WebSocket link to the management server.

Development and Operation of Central Management Server for Data Scrapers

Jan 2023 - Feb 2023

- Built Flask endpoints that manage every scraper from one place.
- Pushed each scraper its targets over that WebSocket connection.
- Hashed login credentials with SHA-256 in transit.
- Split the endpoints into Flask blueprints.
- Put Nginx in front as a reverse proxy.
- Managed HTTPS certificates with Route53 and Certbot.

Development and Operation of BI Tool

Nov 2023 - Nov 2023

- Built the BI tool's server on Flask and NextJS.
- Developed API to deliver social relationship data.
- Rendered social relationships as a directed graph with Cytoscape and SigmaJS.
- Developed API for data labeling.
- Built the components in NextJS on the Carbon Design System.
- Managed HTTPS certificates with Route53 and Certbot.
- Deployed to an EC2 instance (Ubuntu 22.04) with Docker Compose.

Deployment and Operation of Airflow Service

Jan 2023 - Feb 2024

- Wrote Airflow DAGs whose PythonOperators drive the scraper management server on a schedule.
- Wrote DAGs that spin up AWS EMR through boto3 to clean and load the data.
- Ran the DAGs on the Sequential Executor.
- Managed HTTPS certificates with Route53 and Certbot.
- Deployed to an EC2 instance (Ubuntu 22.04) with Docker Compose.

Development of Data Deduplication Feature

Nov 2023 - Feb 2024

- Worked with an ML research engineer on a deduplication system: an IVFFlat index in Faiss-CPU, scoring distance between embedding vectors.
- Extracted image embedding vectors with Facebook's SSCD model.
- Extracted text embedding vectors with OpenAI's Ada embedding model.
- Cut index-creation overhead by 50% with the PostgreSQL pgvector 0.5.0 extension.
- Created pgvector IVFFlat and HNSW indexes.
- Moved the stored vectors from numpy.array to pgvector::vector.

Data Warehouse Schema Design for Efficient Data Management

Feb 2024 - Sep 2024

- Designed a warehouse in DBDiagram around a single-source-of-truth table, to hold the duplicates arriving from many sources.
- Redesigning the schema around S3 as the data lake, since one RDS serving as both source and warehouse had grown expensive.
- Designed DataSource, DataWarehouse, DataMart, and SSoT schemas, encompassing approximately 50 tables.

AWS Service Management

- Implemented user-specific service access control based on IAM Roles.
- Migrated EC2 x86 instance types to arm64 for cost savings.
- Operated Aurora RDS PostgreSQL with separate Read and Write Instances.
- Chose between managed services and self-run EC2 by comparing what each actually cost.

Oct 2021 - Jun 2022

Jung-gu, Seoul

Apr 2022 - Jun 2022

- Feb 2022 - Apr 2022

Feb 2022 - Apr 2022

- Oct 2021 - Jun 2022

Oct 2021 - Jun 2022

- Oct 2021 - Nov 2021

Oct 2021 - Nov 2021

- Split the schema so raw and cleaned data sit apart.
- Cleaned the data in Python with psycopg2

Jan 2019 - Oct 2021

Gangnam, Seoul

- Built CI/CD using Fastlane and Github Actions.
- Built Artwork Hanger, an ARKit feature that hangs a piece on the viewer's own wall.
- Implemented Push Notification feature.
- Built the API layer on Alamofire to call the backend.
- Integrated Unity's Google Cardboard VR through a Unity bridge.
- Implemented asset download from AWS S3 for the VR mode.
- Built a download service that handles several transfers at once.
- Adopted ReactorKit for reactive programming and built out the Model / View / Reactor architecture around it.

- Established unit test environment and progressively expanded coverage.
- Wrote the Auto Layout in SnapKit.
- Built the image view on Kingfisher.
- Developed Pinterest-style CollectionView.

Jan 2018 - Dec 2018

Yeongdeungpo, Seoul

Jan 2018 - Nov 2018

- Jan 2018 - Nov 2018

Jan 2018 - Nov 2018

- Jan 2018 - Nov 2018

Jan 2018 - Nov 2018

- Reused the existing encryption module to implement login.
- Implemented scraping that walks a page's pagination.

Education

Coursera <u>Convolutional Neural Networks</u>	Feb 2023
Coursera <u>Structuring Machine Learning Projects</u>	Nov 2022
Coursera <u>Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization</u>	Oct 2022
Coursera <u>Neural Networks and Deep Learning</u>	Sep 2022
Engineer Information Processing Human Resources Development Service of Korea	Nov 2017
Languages • English: AL (OPIC) • Japanese: Conversational • Korean: Native	
Korea Polytechnic University B.S., Computer Engineering (GPA: 3.4 / 4.5) (Major GPA: 3.73 / 4.5)	Mar 2010 - Feb 2018 Siheung, Gyeonggi-do
• Relevant Coursework: Data Structures, Algorithms (C), Operating System, Computer Architecture, Cloud Computing • <u>[Probono] Furniture Arrangement System Using Augmented Reality for the Socially Vulnerable</u>	

Other Activities

Guryong Village Coal Briquette Volunteering	Jan 2018
Participated in the Let us go iOS Hackathon Built a layer that returns OpenAI API responses in a fixed recipe format	Jun 2024
Republic of Korea Army, Sergeant (Professional NCO) Honorable Discharge. Created an XLSX payroll calculator using Visual Basic Macro	Jun 2011 - Sep 2013

In-House MLOps Platform on Bare-Metal Kubernetes (Pulumi IaC)

- **Core Problem & Goal:** Build an in-house **MLOps platform** from scratch on bare-metal resources so ML researchers can submit distributed training jobs and data engineers can run lakehouse / analytics workloads reliably. The goal was to operate **storage, catalog, lineage, orchestration, distributed training, and observability consistently in a single cluster**, without depending on external managed services.
- **My Role:** Sole owner across the full stack **designing, building, and operating** every layer on a multi-node kubeadm cluster (GPU / storage / CPU node pools) - IaC, networking, HA, lakehouse, orchestration, distributed training, observability.
- **Problem-Solving Process:**
 - * **Trade-off (Managed vs. Self-hosted):** Managed K8s / managed lakehouse offers a fast start, but given internal resource and security requirements plus long-term cost, **bare-metal self-hosted** was the superior choice. Accepted higher up-front investment to keep every layer in-cluster.
 - * **Implementation (How):** Modularized infrastructure into **12 Pulumi stacks** (platform → base → storage → catalog → lineage → orchestration → staging → compute → mlops → cicd → identity → devenv) with explicit cross-stack references for reproducible single-command deploys. Built networking on **Cilium CNI** (KPR + Gateway API + LB-IPAM + L2 Announcements); exposed *.<internal>.* hostnames via a single Gateway with in-cluster CA HTTPS.
- **Results & Impact:**
 - * **Reproducible IaC:** Infrastructure changes are tracked via PRs; new environments (e.g., separate staging) can be reconstituted instantly from stack combinations.
 - * **Unified operations:** All hostnames exposed through one Gateway centralizes certificate and routing management, and minimizes external dependencies.

Active-Active HA Architecture for SPoF Components (CoreDNS, Airflow scheduler)

- **Core Problem & Goal:** Training data pipelines must **keep running through single-node failures**. Critical in-cluster components (CoreDNS, Airflow scheduler) running as single instances meant any node fault could halt the entire workload.
- **My Role:** Identified SPoF candidates and designed / introduced **multi-node active-active redundancy policies** per component; codified operations into runbooks.
- **Problem-Solving Process:**
 - * **Why active-active:** Active-passive failover has unavoidable detection / failover lag, and our training pipelines have **expensive catch-up costs after a restart**. Chose multi-instance **active-active** even at the cost of more operational complexity.
 - * **Implementation (How):** Ran SPoF components (CoreDNS, Airflow scheduler) as multi-replica deployments; applied **PriorityClass / PodDisruptionBudget (PDB)** policies cluster-wide so at least one replica stays alive across node drains / upgrades. Documented operational procedures (drain order, scheduler-restart checks, lakehouse integrity checks) into **runbooks** so the next on-call engineer can follow the same path immediately.
- **Results & Impact:**
 - * **Uninterrupted operations through node failure:** Training DAGs and dataset sync continue through single-node faults and routine upgrades.
 - * **Operational knowledge captured:** Runbooks shorten the learning curve for hand-offs and let recurring incidents be resolved with minimal lead time.

SeaweedFS + Apache Iceberg Lakehouse (Bronze / Silver / Gold Medallion)

- **Core Problem & Goal:** Unify raw ingestion, refinement, and serving of training / analytics data in a single lakehouse. The goal was to assemble an **on-prem S3-compatible distributed storage + table format** lakehouse without external cloud dependency.
- **My Role:** Full ownership of the lakehouse layer - storage selection & provisioning, Iceberg medallion architecture, and ad-hoc query enablement.
- **Problem-Solving Process:**
 - * **Trade-off (MinIO vs. SeaweedFS):** Both are S3-compatible, but the need to manage hot / cold data efficiently across **NVMe / HDD disk tiers** and multi-node replica volume groups was a more natural fit for **SeaweedFS**.

- * **Implementation (How):** Built an **Apache Iceberg lakehouse** on SeaweedFS with a **Bronze / Silver / Gold medallion architecture** in production. To satisfy ad-hoc analytics needs, enabled direct SQL access to Iceberg via **PostgreSQL + pg_duckdb**, and deployed an analyst-facing **DuckDB UI as an in-cluster Pod** (nginx basic auth + automatic token injection) - collapsing the previous "write pyiceberg code" step into a one-line browser SQL query.
- **Results & Impact:**
 - * **Faster analytics cycles:** Analysts no longer need to set up pyiceberg / Spark environments - they run ad-hoc SQL directly in the browser.
 - * **Storage cost efficiency:** Hot data lands on NVMe, cold archival data on HDD tiers, balancing cost and performance in a single cluster.

Airflow 3.x + dbt + Cosmos Orchestration / Transformation Pipeline

- **Core Problem & Goal:** Operationalize Silver / Gold transformations plus Iceberg compaction and integrity verification as **standardized workflows**. Run dbt models naturally inside Airflow and separate prod / staging environments so **minor / patch upgrades ship without release risk**.
- **My Role:** Owned the entire path - Airflow Helm deployment, custom image, dbt + Cosmos integration, environment split, and upgrade standardization.
- **Problem-Solving Process:**
 - * **Why KubernetesExecutor:** Per-task pod isolation simplifies resource management and avoids dependency conflicts. Bundled the in-house dbt project + internal SDK into the same image so **every task runs against an identical dependency graph**.
 - * **Implementation (How):** Deployed Airflow 3.x via **Helm + KubernetesExecutor** on split prod / staging stacks, with **git-sync** CD and a custom image bundling dbt + Astronomer Cosmos + internal SDK. Wrapped Silver / Gold dbt models into Airflow TaskGroups; automated Iceberg compaction and integrity-verification DAGs. Standardized all changes to the **staging-first → prod-sequential** pattern.
- **Results & Impact:**
 - * **Automated transformation pipeline:** Integrity / compaction tasks moved from manual operations into DAGs, removing room for human error.
 - * **Safe upgrade path:** Minor upgrades of Airflow, dbt, or dependencies can ship without downtime via staging-first validation.

Distributed Training Across Mixed GPU Models with KubeRay

- **Core Problem & Goal:** The cluster mixes **GPU nodes of different models**; ML researchers should be able to **submit distributed training jobs without re-tooling their environment** every time.
- **My Role:** Ray cluster setup, unified training image, GPU-type-aware worker group policy, and automated in-house SDK installation pipeline.
- **Problem-Solving Process:**
 - * **Why a unified image:** Letting users set up environments per node causes recurring CUDA / dependency conflicts. **Unifying the worker image** and splitting only by GPU type via worker groups minimized operational cost and user friction.
 - * **Implementation (How):** Configured **KubeRay** over GPU nodes of differing models with a single worker image (CUDA 12.8 · Python 3.12 · PyTorch · ONNXRuntime · Triton); split worker groups by GPU type and automated internal training SDK installation so researchers submit jobs without code changes.
- **Results & Impact:**
 - * **Environment setup cost eliminated:** The setup step before distributed training disappeared; researchers focus on the code.
 - * **Mixed GPUs put to work:** GPUs of different models run side by side inside a single Ray cluster, improving overall cluster utilization.

Integrated Catalog & Observability with DataHub / MLflow / Prometheus

- **Core Problem & Goal:** Dataset catalog, ML experiments / model registry, and cluster metrics were scattered. Needed a **unified observability layer where data flow and model lineage can be traced in one place**.
 - **My Role:** DataHub deployment + OpenLineage integration, MLflow operation, Prometheus / Grafana cluster-metric standardization, self-hosted Docker Registry + GitHub Actions CI/CD.
 - **Implementation (How):** **DataHub** for dataset catalog + **OpenLineage**-driven Airflow / dbt lineage collection; **MLflow** for experiments and model registry; **Prometheus + Grafana** for cluster metrics; **self-hosted Docker Registry + GitHub Actions** for unified image build and deploy.
 - **Results & Impact:**
 - * **Data + model lineage visibility:** Flow from dataset to model is traceable in a single view; change-impact analysis moves from manual tracing to catalog search.
 - * **Simplified image supply chain:** Infra and application image build / deploy unified into a single CI workflow with the self-hosted Registry.
-

Integrated Data Platform for Art Investment Analysis

- **Core Problem & Goal:** All art market data, including **artists, artworks, auctions, social networks, institutions, and artist CVs**, was fragmented across multiple sources. The final goal was to **build a reliable, integrated data pipeline** to encompass all this data, utilize it as an AI and statistics-based analysis platform, and **contribute to securing Series B funding**.
- **My Role:** As the Data Engineering Team Lead, I led the entire project, from the data pipeline's **architecture design to its implementation, operation, and the development of an LLM agent prototype**.
- **Problem-Solving Process:**
 - * **Trade-off Analysis (Pandas vs. PySpark):** Since the project's final goal was to collect all data, I **designed the architecture assuming** the high probability that the data scale would grow to hundreds of millions of records in the future. To secure future **scalability and reliability** over current development speed, I made the decision to proactively **adopt PySpark**, despite the higher initial overhead.
 - * **Specific Implementation (How):** Implemented the data purification and loading pipeline by having the **Python files that define each data processing stage included and executed sequentially as EMR Steps** when the AWS EMR cluster is instantiated. Within this pipeline, I built a system that utilized Facebook's **SSCD** and OpenAI's **Embedding** models and filtered duplicate data using **PGVector's HNSW** index.
- **Result & Impact:**
 - * **Business Impact:** The high-quality, integrated data platform that was built became a core company asset and played a **decisive role in successfully securing Series B funding**.
 - * **Technical Achievement:** By proactively adopting PySpark, I established the foundation for a scalable architecture that could stably operate the pipeline without a separate migration, even as data volume surged.

[go back](#)

Central Management & Monitoring Server for Large-Scale Data Scrapers

- **Core Problem & Goal:** As we operated **over 50 scraper instances simultaneously** for data collection, managing the status of each instance individually consumed significant resources. The goal was to **maximize the stability and operational efficiency of the data collection pipeline** by building a server to centrally control and monitor all scrapers in real-time.
- **My Role:** I was responsible for the **entire End-to-End development**, from the server's architecture design to API endpoint development, WebSocket communication, and Nginx/HTTPS deployment.
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Judged that a **WebSocket** approach, which maintains a persistent bidirectional channel, was more efficient for real-time monitoring than HTTP Polling, as it reduces server load and propagates status changes instantly.
 - * **Trade-off Analysis (Trade-off):** Considered 'Flask' and 'FastAPI'. As the project's core need was stable real-time communication over complex async logic, I **chose 'Flask'**, which I was more familiar with and had a more mature library ecosystem (Flask-SocketIO), to prioritize development speed and stability.
 - * **Specific Implementation (How):** Developed scraper control APIs based on '**Flask**' and modularized the code with '**Flask-Blueprint**'. Implemented real-time status/metric exchange via '**WebSocket**'. Secured login credentials via '**SHA-256**' encryption and applied an HTTPS certificate from '**Certbot**' using '**Nginx**' as a reverse proxy.
- **Result & Impact:**
 - * **Maximized Operational Efficiency:** Dramatically reduced operational resources by enabling central management of over 50 scraper instances via a single API, which were previously managed manually.
 - * **Improved Stability & Reliability:** Significantly improved the data collection pipeline's overall stability and reliability by enabling immediate identification and remote restart of failed instances.

[go back](#)

Full-Stack BI Tool for Social Network Analysis and Data Purification

- **Core Problem & Goal:** The complex graph data from ML models was difficult for non-developer art experts to analyze. An efficient tool was needed for them to visually explore the network and simultaneously label data to improve its quality.
- **My Role:** Led the **entire Full-Stack development**, from data structure design and API development (**Flask**) to frontend (**Next.js**), visualization (**Cytoscape/SigmaJS**), and final deployment (**Docker, Nginx**).
- **Problem-Solving Process:**

- * **Technical Consideration (Why):** The key challenge was smoothly rendering large-scale graphs in a browser. I initially used **Cytoscape.js**, but migrated to the WebGL-based **Sigma.js** to improve performance when the number of nodes and edges grew.
 - * **Trade-off Analysis (Trade-off):** Chose IBM's **Carbon Design System** over more common UI libraries. Despite an initial learning curve due to fewer resources, I judged it would improve long-term productivity with its specialized components for data visualization dashboards.
 - * **Specific Implementation (How):** Developed RESTful APIs for social network and labeling data with **Flask**. Built the frontend with **Next.js** and UI components with **Carbon Design System**. Finally, containerized and deployed the Nginx, Next.js, and Flask servers on an EC2 instance using **Docker Compose**.
 - **Result & Impact:**
 - * **Improved Work Efficiency:** Dramatically reduced the time required for analysis by enabling art experts to directly explore social network data through the BI tool, instead of requesting it from developers.
 - * **Improved Data Quality:** Provided a feature for experts to directly correct and label data, which contributed to improving the quality of training data for the ML model and created a foundation for a virtuous cycle of data purification.
- [go back](#)

Cost-Effective Airflow Implementation for Data Pipeline Automation

- **Core Problem & Goal:** The manual operation of the data collection and purification pipeline was inefficient and prone to errors. The goal was to build a stable scheduling system to **automate execution, monitoring, and reprocessing** while **minimizing operational costs**.
 - **My Role:** I was responsible for the **entire workflow automation process**, including Airflow infrastructure design, EC2/Docker-based deployment, and authoring core DAGs.
 - **Problem-Solving Process:**
 - * **Trade-off 1 (Infrastructure):** Chose to **deploy Airflow directly on EC2 with Docker Compose**, which required more initial setup effort, to reduce infrastructure costs by over 90% compared to the expensive **AWS Managed Airflow (MWAA)**. This was also driven by MWAA's version constraints which did not support my preferred **TaskFlow API**.
 - * **Trade-off 2 (Executor):** Chose the simplest and most stable **Sequential Executor** to minimize infrastructure complexity, as the project's core need was sequential stability over the parallel processing offered by the more complex **Celery Executor**.
 - * **Specific Implementation (How):** Deployed Airflow on EC2 using **Docker Compose**. Authored a scraping DAG that calls the scraper management server's API via a **PythonOperator**, and a data purification DAG that triggers an EMR job using **boto3**.
 - **Result & Impact:**
 - * **Maximized Operational Efficiency:** **Fully automated** the daily manual data tasks, reducing the time spent on related operations by over 95%.
 - * **Improved Stability & Observability:** Enabled central monitoring of all pipeline statuses and logs via the Airflow UI, significantly reducing the time to identify and respond to failures.
- [go back](#)

Embedding-based Image/Text Deduplication System Development and Optimization

- **Core Problem & Goal:** The initial deduplication system had significant **unnecessary overhead**, as it required downloading embeddings from the DB, converting data types, and creating an index externally. The goal was to **optimize the pipeline** and improve data quality by handling all logic within the database.
- **My Role:** Analyzed the pipeline overhead of the initial system developed by an ML engineer and **led the design and implementation of a new, optimized architecture based on PGVector**.
- **Problem-Solving Process:**
 - * **Technical Consideration (Problem with old way):** The initial **'Faiss-CPU'**-based system was inefficient as it required pulling data from PostgreSQL and **converting incompatible data types** before creating an index locally.
 - * **Specific Implementation (How):** To solve this overhead, I changed the process to handle all logic inside the DB. I performed a data migration from the existing **'Numpy' array to the 'pgvector::vector' type** and rebuilt the system by creating **'PGVector's 'IVFFlat' and 'HNSW' indexes**.
- **Result & Impact:**
 - * **Improved Operational Efficiency & Cost Savings:** By completely eliminating the unnecessary data transfer and conversion steps, I **reduced the overall pipeline overhead by over 50%** and simplified the architecture.
 - * **Improved Data Quality:** The optimized system dramatically improved data consistency and reliability, playing a key role in building a high-quality Price Database.

[go back](#)

Data Warehouse Redesign for Cost Optimization and Data Reliability

- **Core Problem & Goal:** The existing single RDS-centric architecture suffered from severe **data duplication and unnecessary cost issues**. The goal was to improve data reliability and reduce operational costs by separating storage layers for each data processing stage and introducing an **architecture based on a Single Source of Truth (SSoT)**.
- **My Role:** Analyzed the problems in the data pipeline and **led the redesign of the data warehouse structure** to improve cost-efficiency and data reliability.
- **Problem-Solving Process:**
 - * **Trade-off Analysis (RDS vs. S3 Data Lake):** While the existing RDS structure was simple to develop initially, its long-term cost and scalability issues were clear. I made the decision to **introduce the low-cost S3 as a data lake**, despite the initial migration costs, to secure long-term storage cost savings and management flexibility.
 - * **Specific Implementation (How):** Visually designed DataSource, Data Warehouse, and Data Mart layers using the DBDiagram tool. **Designed approximately 50 table schemas, including an SSoT table** to centrally manage duplicate data.
- **Result & Impact:**
 - * **Established Foundation for Cost Savings:** By introducing an S3 data lake, I established a foundation that could dramatically **reduce anticipated storage costs** compared to the previous architecture.
 - * **Improved Data Reliability:** Established a structure that can systematically manage data duplication and improve consistency and reliability through the **SSoT table design**.

[go back](#)

MLOps Engineer, MLOps at Eazel Inc.

Oct 2021 - Jun 2022

Jung-gu, Seoul

MLOps-based Instagram Network Analysis and Investment Insight PoC

MLOps Engineer

- **Core Problem & Goal:** To objectify/quantify artist market influence, which previously relied on experts' subjective judgments, using Instagram network data. The final goal was to prove growth potential with data and **succeed in securing Series A funding**.
- **My Role:** As an MLOps Engineer, I led the construction of a **reproducible End-to-End pipeline** for the PoC, from data collection, **data/model versioning using DVC**, processing inputs for various models, to K-Means/PCA analysis and result visualization.
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Lacking deep knowledge of the machine learning models themselves at the time, I chose to collaborate with an ML Research Engineer, allowing us to focus on our respective areas of expertise. My colleague handled various graph embedding models like **Node2Vec and GAT**, while I focused on the data pipeline and subsequent analysis/visualization.
 - * **Specific Implementation (How):** Introduced **DVC**, which integrates with Git, to track and version the model's input data and artifacts by linking them to Git commit hashes, thereby **ensuring the reproducibility** of experiments. Afterwards, I took the extracted embedding vectors as input and analyzed artist groups using **K-Means clustering** and **PCA**. Finally, I implemented an interactive network visualization dashboard with these analysis results using **Next.js and Cytoscape.js**.
- **Result & Impact:**
 - * **Business Impact:** Proved artist growth potential through the analytics from the developed visualization dashboard, making a **decisive contribution to successfully securing Series A funding**.
 - * **Personal & Technical Growth:** Through this collaborative experience, I gained a deep practical understanding of building machine learning pipelines, and it became the decisive catalyst for my career transition to a Data Engineer.

[go back](#)

Data Scraper Development and Enhancement for Bypassing Instagram Bot Detection

- **Core Problem & Goal:** To solve the problem of being unable to continuously collect data due to Instagram's strong **Bot Detection system**. The goal was to develop a robust scraper that could stably bypass bot detection and collect large-scale social relationship data **without interruption**.
- **My Role:** Responsible for researching, prototyping, and implementing the final solution, exploring various technical approaches (**Selenium, requests, Instagrapi**, etc.) to solve the bot detection problem.

- **Problem-Solving Process:**
 - * **Trade-off Analysis (Server vs. Mobile Environment):** Confirmed that using a Python-based scraper in a server environment was ultimately blocked by bot detection. Based on the hypothesis that the 'server environment' itself was a major factor in detection, I chose a fundamental solution: to **personally migrate the Python library to Swift**, despite the high initial cost, to emulate requests from a mobile environment.
 - * **Specific Implementation (How):** **Personally ported the core logic of 'Instagrapi' from Python to Swift**, developing a data scraper that runs in an iOS mobile environment. During development, I fixed bugs found in the library and **contributed directly to the open-source project**.
 - **Result & Impact:**
 - * **Core Problem Solved:** Succeeded in **stably collecting large-scale social relationship data**, which was impossible with previous methods, providing the foundation for the 'MLOps Analysis PoC' project.
 - * **Demonstrated Problem-Solving Ability:** Solved a persistent external system limitation through the creative approach of porting a framework to a different language.
- [go back](#)

Initial Data Pipeline Construction using PostgreSQL Schema Separation

- **Core Problem & Goal:** Needed to efficiently separate and manage collected raw data and purified data. In a situation where building a separate warehouse was difficult, the goal was to build a practical pipeline that could **preserve data integrity and reliably serve data within a single DB**.
 - **My Role:** Developed the data purification logic and **personally designed and implemented an architecture that separates data layers by separating schemas within PostgreSQL**.
 - **Problem-Solving Process:**
 - * **Trade-off Analysis (DB Separation vs. Schema Separation):** Physically separating the database had the disadvantage of doubling costs and management points. Instead, I chose the **PostgreSQL schema separation method**, which allows for logical separation with no additional cost, securing practicality and cost-efficiency.
 - * **Specific Implementation (How):** Created `raw_data` and `purified_data` schemas within **PostgreSQL**. Built a pipeline that performs data purification logic using **Python's psycopg2 library** and stores the data in tables within their respective schemas.
 - **Result & Impact:**
 - * **Ensured Data Integrity:** Established an environment where raw data is preserved without changes and analysts can safely use only purified data, reducing data management complexity.
 - * **Paved the Way for Growth:** This experience provided a foundational understanding of the need for data warehouses, paving the way for designing more sophisticated data architectures later on.
- [go back](#)

iOS Developer, iOS Team

Jan 2019 - Oct 2021
Gangnam, Seoul

iOS Application Development & Deployment

iOS Developer

- **Core Problem & Goal:** To develop a native iOS application for displaying art content, featuring VR capabilities, asset downloading, and AR functionalities.
- **My Role:** Led the development of the app's backend layer, including the API Client. Implemented detail views and view controllers for artworks, artists, and exhibitions. Developed key features including a Google Cardboard VR view via a Unity Bridge, a download service for VR assets, and an AR-based "Artwork Hanger" feature using ARKit.
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Faced with upcoming complex features like VR asset downloading and Unity integration, I anticipated that the existing MVC structure would lead to increased complexity and reduced maintainability. The goal was to adopt a reactive architecture for clearer and more predictable state management.
 - * **Trade-off Analysis (Trade-off):** I first considered **implementing the MVVM pattern manually**. However, I judged that this would require significant initial development resources for defining protocols and data binding logic. As an alternative, I reviewed the reactive programming framework, **ReactorKit**. I was impressed that ReactorKit enforces a clear, 3-step unidirectional data flow — **User Input (Action) -> Business Logic Processing (Mutate) -> Applying State Change (Reduce)** — which fundamentally prevents the complexity of manual implementation and makes the code flow very easy to understand. I concluded that despite the initial learning curve, adopting ReactorKit was more advantageous for long-term maintainability and development efficiency.

- * **Specific Implementation (How):** Re-architected the app based on ReactorKit. Unified the data flow by defining all user inputs as 'Actions' and state changes as 'State', and integrated the download service using an AWS S3 client into the Reactor logic.
- **Result & Impact:**
 - * **Improved Development Efficiency:** After getting used to the framework, **the process of defining logic according to the Action-Mutate-Reduce structure became very easy and clear.** This led to improved development speed for new features and maintained code consistency.
 - * **Enhanced Stability & Maintainability:** The clear structure reduced the occurrence of state management-related bugs, and shortened the time it took for new team members to understand the codebase.